Performance Analysis of Apache ZooKeeper's Write Pipeline
Software Performance Engineering — Project Stage II

Gil Jorge (110062), João Palma (109242), and Gabriel Petti Tavares (120691)

Instituto Superior Técnico, Universidade de Lisboa — Group 18
gil.jorge@tecnico.ulisboa.pt, joaofontepalma@tecnico.ulisboa.pt,
gabriel.p.tavares@tecnico.ulisboa.pt

Abstract. We study the write pipeline of Apache ZooKeeper 3.9, a coordination service in which reads are served locally by any server while every write is serialized through one leader and a quorum. Stage II systematizes the evaluation on Google Compute Engine, with the servers and the load generator on separate machines. We (i) run a 2^{6-1} fractional factorial experiment over six factors (number of servers, write ratio, number of clients, `forceSync`, payload size and log disk) with two replicates and a randomized, blocked run order; (ii) fit one Universal Scalability Law curve, with replicates, bootstrap confidence intervals and a test of whether the coherency term is justified; and (iii) follow a write through the pipeline with ZooKeeper's own per-stage counters and five eBPF probes (fsync latency, block-I/O latency, run-queue latency, TCP retransmissions and the CPU used by each pipeline thread). Initially the load generator logged every operation, which throttled it and made the contention coefficient look independent of the number of servers, and the reported effect of network delay compared against the wrong baseline. Across the factorial experiment the share of writes explains 60% of the variation of throughput (a write costs about as much as ten reads) and the number of clients 94% of the variation of the write tail latency; disabling `fsync` raises throughput by 39% at low load but lowers it by 17% at high load, where the log-sync stage already writes 17 requests per `fsync`. The scalability curve is described by contention alone ($\alpha = 0.039$; a coherency term is not supported), and ZooKeeper's per-stage counters and the probes place that contention in the leader's request pipeline rather than in the disk.

Git Commit Identifier: `9a52eabcfe0019b8ab4ac96fb7bb2195694d70a0`

1 Introduction

Apache ZooKeeper is an open-source coordination service used by systems such as Kafka, HBase and Hadoop for leader election, configuration management and distributed locking [1, 3]. It has a clear client-server split, is genuinely distributed, and has a published replication protocol, Zab [2]. We chose it because its read/write asymmetry gives a clean hypothesis: reads are answered by any server and should scale with the number of servers, whereas every write passes through a single leader and must reach a majority, so writes should not.

Stage I measured throughput against the number of concurrent clients on one laptop and fitted the Universal Scalability Law (USL) [6, 7]. Stage II asks for a more systematic evaluation. Our contributions are:

- a corrected baseline: we found and fixed a client-side artifact in the Stage I measurements (Section 3) and show how large run-to-run variation is on shared hardware;
- a 2^{6-1} fractional factorial experiment on Google Compute Engine that ranks the factors and interactions that drive throughput and write tail latency (Sections 5 and 6.1);
- a single, well-supported USL fit, with the time a write spends in each pipeline stage measured at every load level, so that the coefficients can be tied to stages of the pipeline (Section 6.2);
- five eBPF probes, four of them our own, that observe the log-sync stage, the disk, the scheduler, the network and the CPU of each pipeline thread from inside the kernel (Section 6.3).

2 System Description

2.1 Data model, consistency and the two request pipelines

ZooKeeper stores a small tree of data nodes (znodes) entirely in memory and persists changes to a transaction log. A group of N servers elects one *leader*; the others are *followers*. A client keeps a session with one server. Writes are linearizable through Zab; reads are sequentially consistent per session and may be slightly stale, which is what lets every server answer them alone [1]. Figure 1 shows both pipelines.

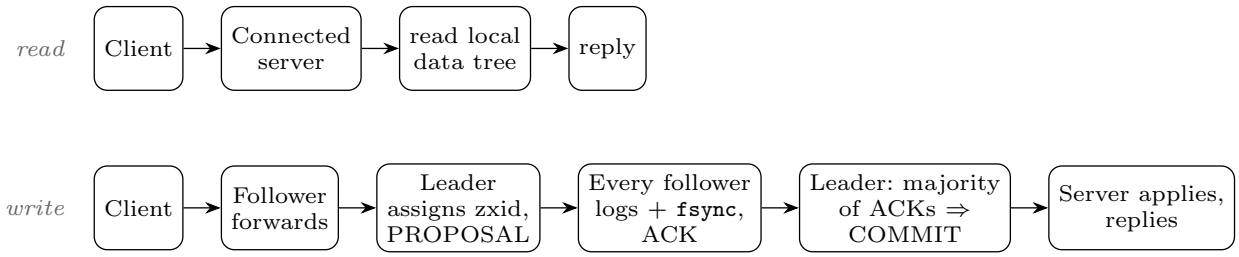


Fig. 1. Read and write pipelines. A write passes the leader and waits for a majority of followers that have logged it to disk; a read never leaves the server the client is attached to.

Inside a server a request goes through a chain of processors. The ones that matter here are request preparation (assigns the zxid and validates the operation on the leader), the log-sync stage (appends to the transaction log and groups `fsync` calls) and the commit stage (applies the operation once it is committed). Request preparation and log sync are each handled by a single thread, which makes them the natural candidates for the serialized fraction α of the USL. With $N = 2f + 1$ servers the group of servers tolerates f failures and keeps serving while a majority is reachable, choosing consistency over availability otherwise [2].

2.2 Why the write pipeline is the object of study

A read costs one hop and no coordination; a write needs a forward to the leader, a proposal, a disk write and acknowledgment at every follower, and a commit. Throughput of reads therefore grows with the number of servers, while the cost of a write grows with it. The experiments of this report ask which stage of the write pipeline limits scalability and which of the cost terms of the USL it corresponds to.

3 Stage I

The load generator logged every operation. The same configuration (3 servers, 20 clients, 30% writes) reached 6,364 ops/s in the Stage I sweep but 14,832 ops/s when the sweep was repeated with a quiet client. A controlled comparison showed that the jar used for the sweep printed about 145,000 lines in a 15 s run, roughly one per operation: the shaded jar contained Logback, whose default configuration logs the ZooKeeper client at DEBUG level, so all client threads queued on the output. With the output redirected to a file, 15 interleaved runs of five variants gave 6,300–12,000 ops/s, and neither the JVM heap flags nor the metrics collector made a consistent difference. The fix is a one-line Logback configuration that turns logging off, and the experiment runner now aborts a run if the client prints more than a few lines. The Stage I fits gave the same $\alpha \approx 0.23$ for 1, 3 and 5 servers, and we concluded that the contention was server-side but not specific to Zab. A client-side serialization point looks exactly like that, because it does not depend on the number of servers; the constant α was most likely an artifact of the client. With the quiet client the fitted α was 0.025, 0.088 and 0.112 for 1, 3 and 5 servers, and the standalone server no longer saturated within 80 clients (29,000 ops/s).

Run-to-run variation on a laptop is large. In the same comparison, repetitions of an identical variant differed by up to a factor of two (6,285 against 12,043 ops/s), more than any difference between the variants. The Stage I sweep used one run per point in order of increasing load, so drift and load were confounded, and the differences between the three fitted curves are within this noise. Stage II therefore uses dedicated virtual machines, replicates and a randomized run order.

4 Testbed and Methodology

Deployment. All experiments of Stage II run on Google Compute Engine in one zone (europe-west1-b). The load generator runs on its own virtual machine (n2-standard-8: 8 vCPUs, 32 GB) and the ZooKeeper servers each on a separate machine (n2-standard-2: 2 vCPUs, 8 GB), so that client and servers do not compete for CPU, which was a limitation of Stage I. All machines run Debian 12 (kernel 6.1) and OpenJDK 17, with boot disks of type pd-standard. ZooKeeper 3.9.3 runs natively (not in a container) with a 1 GB heap, `tickTime`=2000 ms and no client-connection limit. Each server has two extra disks, pd-balanced and pd-ssd, and the transaction log (`dataLogDir`) is placed on one of them according to the factor *log disk*; snapshots stay on the boot disk. Before each block of runs the servers are stopped, their data and logs are wiped and they are restarted with the levels of the block. The deployment is scripted (Compute Engine only, no Kubernetes) and the servers are configured over ssh.

Load generator. The benchmark is a multi-threaded Java client built on an `ExecutorService` with one ZooKeeper session per concurrent client; the connect string lists all servers, so sessions spread across the servers. Each client is a closed loop of synchronous calls with no think time: it picks a znode uniformly from a pool of 200 znodes created before the run, and issues a `setData` with probability equal to the write ratio and a `getData` otherwise. All sessions are connected before a barrier releases them together, so connection time never enters the warm-up or the measurement. Operations that start during the 5 s warm-up are executed but not recorded. Latencies are recorded per thread, merged at the end, and percentiles are computed exactly by sorting. Throughput is the number of successful operations divided by the measurement window; failed operations are counted by exception type and excluded.

Run protocol and checks that the client is not the bottleneck. During every run the client machine samples its own CPU utilization (`vmstat`, once per second, over the measured window); a run with a client above about 70% busy would measure the client and not ZooKeeper. The runner also aborts if the client prints more than a few lines of output (Section 3), and verifies on every server, before the first run of each block, that the server-side factors were really applied: the `forceSync` flag in the arguments of the running JVM and the directory of the transaction log reported by the server itself.

ZooKeeper's own view of the pipeline. ZooKeeper 3.9 keeps a summary per pipeline stage (count and sum of, for example, the time a request waits for the request-preparation thread, the time it waits for and spends in the log-sync stage, the number of requests written per `fsync`, the `fsync` time and the time until a quorum has acknowledged a proposal) and reports them through the `mntr` command. Around every run of the USL sweep and of the probe runs we take a snapshot of every server; the difference of two snapshots divided by the difference of the counts is the mean of that stage over that run alone. These timers have a resolution of 1 ms (the `fsync` time is truncated to whole milliseconds), which is why the `fsync` is also measured with eBPF in microseconds.

Evaluation process. The evaluation is three experiments, all scripted (`gce/vm_jobs.sh`) and all with their raw data in the repository: (1) the factorial experiment of Section 5, 64 runs in a randomized, blocked order that is fixed before the first run (`factorial/plan_gce/plan.csv`); (2) one USL curve, 13 load levels with three replicates in random order, with the per-stage counters around every run (Section 6.2); (3) the probe runs, one per combination of `forceSync` and log disk, each paired with an identical run without probes to measure their overhead (Section 6.3). The analysis scripts were validated on synthetic data with known effects before they were applied to measurements.

Threats to validity. Virtual machines on a shared cloud have neighbours we cannot control; replicates and randomization are our defence. The servers are in one zone, so inter-server round trips are far shorter than in a geo-replicated deployment. Disabling `forceSync` is not a safe production setting and is used only as an experimental probe.

5 Experimental Design

5.1 System and workload parameters

Table 1 lists the parameters that can influence the system under test. The factors of the experiment are chosen among them; the others are fixed and reported so that the experiment can be repeated.

5.2 Metrics

The primary metric is *throughput* (successful operations per second), because the USL is a model of throughput. The secondary metric is the *99th percentile write latency*: coordination services sit on the critical path of other systems, so the tail of the write path matters as much as its rate. Read latency is recorded but is not a response of the design, because reads never leave the server they are sent to. The mean latency is not a response either: in a closed loop without think time, Little's law gives a mean response time of $R = N/X$ for N clients, so it carries no information beyond the throughput, whereas the 99th percentile does. Two further quantities are recorded in every run as validity checks, not responses: the number of failed operations and the CPU utilization of the load generator.

Table 1. System and workload parameters. The six in bold are the factors of the experiment (their letter in the last column); the others are fixed at the value shown. Defaults are those of ZooKeeper 3.9 [4].

Parameter	Value(s)	Role
System		
Number of voting servers	3, 5	A
`forceSync` (fsync of the transaction log)	yes, no	D
Log disk (device of `dataLogDir`)	pd-balanced, pd-ssd (20 GB each)	F
Observers (non-voting servers)	0	fixed
Snapshot device (`dataDir`)	boot disk (pd-standard)	fixed
`snapCount`, `preAllocSize`	100,000, 64 MB (defaults)	fixed
Log batching (`maxBatchSize`, `flushDelay`)	1000, 0 (defaults: flush when the queue is empty)	fixed
`globalOutstandingLimit`, `maxClientCnxns`	1000 (default), no limit	fixed
`tickTime`, `initLimit`, `syncLimit`	2000 ms, 10, 5	fixed
`leaderServes` (leader accepts clients)	yes (default)	fixed
Server JVM	JDK 17, 1 GB heap, G1 collector	fixed
Versions	ZooKeeper 3.9.3; client library 3.9.0	fixed
Machines and network	see Section 4; one zone	fixed
Workload		
Concurrent clients (one session each)	10, 160	C
Write ratio (`setData` vs. `getData`)	0.1, 0.5	B
Payload size	128 B, 4 KiB	E
Operations	`getData`, `setData`; no creates, deletes, watches or multi-operations	fixed
Key space and popularity	200 znodes, uniform	fixed
Client model	closed loop, synchronous, no think time	fixed
Session placement	random server of the connect string (all servers)	fixed
Run length	5 s warm-up + 20 s measured; new znode pool per run	fixed
Load generator	one VM, JVM heap 1–2 GB	fixed

Table 2. Factors, levels and justification. "Hard" factors require reconfiguring the group of servers.

Factor	−1	+1	Hard	Why
A ensemble size	3	5	yes	quorum size: proposals and ACKs per write; the two usual production sizes
B write ratio	0.1	0.5		share of requests that take the write pipeline (reads stay local)
C clients	10	160		below the saturation knee vs well beyond it
D force sync	yes	no	yes	removes the fsync from the write path: tests the log-sync stage
E payload bytes	128	4096		bytes logged, sent to every follower and synced per write
F log disk	pd-balanced	pd-ssd	yes	speed of the device under the fsync (dedicated log disk)

5.3 Factors and levels

Each factor is tied to a hypothesis from Stage I about where the cost of the write pipeline sits (Table 2). The low level of the number of clients (10) lies below the saturation knee measured in Stage I and the high level (160) lies well beyond it; the write ratio 0.1 is close to the production ratio of about 10:1 cited in [5] and 0.5 stresses the write pipeline. Disabling `forceSync` removes the log-sync stage from the write path; it is a controlled probe, not a recommendation. Sets of three and five servers are the two sizes used in practice (they tolerate one and two failures). The payloads bracket small metadata (128 B) and configuration blobs (4 KiB), far below the 1 MB limit of a znode. pd-balanced and pd-ssd are the two Persistent Disk types a dedicated log device would realistically use; Persistent Disk performance grows with the size of the disk and of the machine, so with 20 GB disks on two-vCPU servers these are levels of this testbed rather than of the disk types in general.

5.4 Design

A full 2^6 design needs 64 runs per replicate. We use a half fraction, 2^{6-1}, generating the sixth factor as $F = ABCDE$, i.e. with the defining relation $I = ABCDEF$ [8]. The design has resolution VI: every main effect is aliased only with a five-factor interaction and every two-factor interaction only with a four-factor interaction; three-factor interactions are aliased in pairs. Assuming that interactions of order four and higher

Table 3. Alias structure ($I = ABCDEF$, resolution VI): main effects and two-factor interactions.

Effect	Aliased with	Effect	Aliased with	Effect	Aliased with
A	BCDEF	AC	BDEF	BF	ACDE
B	ACDEF	AD	BCEF	CD	ABEF
C	ABDEF	AE	BCDF	CE	ABDF
D	ABCEF	AF	BCDE	CF	ABDE
E	ABCDF	BC	ADEF	DE	ABCF
F	ABCDE	BD	ACEF	DF	ABCE
AB	CDEF	BE	ACDF	EF	ABCD

are negligible, all six main effects and all fifteen two-factor interactions are estimated cleanly from 32 runs per replicate (Table 3). With two replicates the experiment has 64 runs and 32 degrees of freedom for the error, which gives confidence intervals on every effect. The full sign table is in Appendix A.

Why this fraction, and how the sign table is built. A full 2^6 design would need 128 runs for two replicates; the half fraction needs 64 and loses nothing we intend to estimate, because what it confounds with the main effects and two-factor interactions are interactions of order five and four. A quarter fraction (2^{6-2}, 16 runs per replicate) would have resolution IV: two-factor interactions would be aliased in pairs, and that is exactly where hypotheses H2–H4 sit. The sign table is the full 2^5 table of factors A–E with the column of F computed as the product $ABCDE$ of the other five; every column is balanced and orthogonal to all others (checked by `design.py`). The effect of a factor or interaction is the inner product of its column of signs with the 32 run means, divided by 32; the column of an interaction is the product of the columns of its factors. The plan, with its run order, was generated with a fixed seed before the first run and is part of the repository.

Restricted randomization. Number of servers, `forceSync` and log disk require restarting and reconfiguring the servers. Runs that share the levels of these three factors form a *block* (eight blocks of four runs per replicate, so 16 server rebuilds instead of 64). The order of the blocks is randomized independently in each replicate and the order of the runs inside a block is randomized as well. Because every replicate rebuilds the servers from scratch, restart-to-restart variability enters the pure-error estimate. This is not a full split-plot analysis: a proper one would test the three hard-to-change factors against a separate whole-plot error with fewer degrees of freedom, so their confidence intervals may be somewhat optimistic. We therefore interpret them mainly by their share of the variation and treat only large effects as conclusive.

5.5 Analysis method

Let s_{ij} be the ± 1 entry of the sign table for run i and effect j, $\bar{y}_i$ the mean of the r replicates of run i and $n = 32$ the number of runs. The effects, the allocation of variation and the confidence intervals are

$$q_j = \frac{1}{n}\sum_i s_{ij}\,\bar{y}_i, \qquad SS_j = n\,r\,q_j^2, \qquad SSE = \sum_{i,k}(y_{ik} - \bar{y}_i)^2, \tag{1}$$

$$SST = SSE + \sum_j SS_j, \qquad s_e^2 = \frac{SSE}{n(r-1)}, \qquad q_j \pm t_{[0.975;\,n(r-1)]}\,\frac{s_e}{\sqrt{n\,r}}. \tag{2}$$

The share of the variation explained by effect j is $100\,SS_j/SST$. With 31 effects tested at the 95% level about two false positives are expected, so effects are ranked by their share of the variation and not only by significance. We check the residuals against the predicted value and on a normal quantile plot (Fig. 5 in the appendix); if the ratio between the largest and the smallest response exceeds 10, the analysis is repeated on $\log_{10}$ of the response (a multiplicative model). The analysis code was validated on synthetic data with known effects before it was applied to measurements.

5.6 Hypotheses

We fix the expected outcomes before running, so that the experiment can falsify them.

– **H1 (log-sync stage).** Disabling `forceSync` (D) increases throughput and lowers the write p99 latency; the eBPF fsync probe shows (almost) no fsync calls when D is at its high level. If D has no effect, the contention coefficient α does not come from the fsync.

– **H2 (writes).** The benefit of disabling fsync grows with the write ratio (interaction B×D), since reads never touch the log.
– **H3 (disk).** A faster log disk (F) helps only while fsync is on (interaction D×F).
– **H4 (quorum).** A larger group of servers (A) lowers throughput and raises the write p99 latency, concentrated in writes (interaction A×B).
– **H5 (load).** The number of clients (C) has the largest main effect on throughput, because its low level is below and its high level above the saturation point.

6 Results

6.1 Factorial experiment

All 64 runs completed without a failed operation, and the load generator never exceeded 47% of its CPU (17% on average), so no run measured the client instead of ZooKeeper. Throughput ranged from 5,335 to 81,088 ops/s and the write p99 from 1.2 to 61 ms. Both ratios exceed 10 and the residuals of the additive model fan out with the predicted value, so both responses are analysed on $\log_{10}$, where the residuals are homogeneous and close to normal (Fig. 5); effects are then factors, not differences. The replicates leave only 2.1% (throughput) and 1.3% (write p99) of the variation to the error, so the effects in Table 4 are well above the noise.

Throughput. The write ratio (B) dominates: going from 10% to 50% writes divides throughput by 2.9 (×0.35, 60% of the variation). If a read costs the servers r and a write w, $(0.9r + 0.1w)/(0.5r + 0.5w) = 0.35$ gives $w \approx 10r$: a write costs about as much as ten reads. The number of clients (C) comes second (×1.98, 26%), and 4 KiB payloads (E) cost 26% (×0.74, 5%). The largest interaction is clients × `forceSync` (C×D, 3.6%, Fig. 2): with 10 clients, disabling the `fsync` raises throughput from 12,381 to 17,150 ops/s (geometric means), but with 160 clients throughput is *lower* without it: 26,270 against 31,776 ops/s. At low load every write waits for its own `fsync`; at high load the log-sync stage writes many requests per `fsync` (group commit), so the `fsync` no longer limits throughput, and once the servers are saturated, throughput is set by the CPU each request costs. The probe runs (Section 6.3) show the batching directly: with the `fsync` the log-sync stage writes 16–17 requests per flush, without it 1.9. We read the reversal as the price of losing that batching at saturation: more, smaller flushes and acknowledgment rounds cost the leader more CPU per request. The probes ran at 80 clients, where disabling the `fsync` still gains 8–10%, so they show the batching but not the reversal itself; the explanation is consistent with the data rather than proven by it. The reversal itself is robust: at 160 clients, throughput without `fsync` is 0.79–0.87 of that with it at every level of every other factor. A larger set of servers raises throughput by 14% (×1.14) at both write ratios: the extra servers answer reads, and the larger quorum costs less than they add, even with 50% writes. The log disk (F) has no main effect and appears only in a small E×F interaction (0.5%).

Write tail latency. The p99 of writes is almost entirely a function of the load: 160 clients raise it 11.7-fold (94% of the variation), as expected once the servers are saturated, since by Little's law the time in the system grows with the number of outstanding requests. Every other effect is a correction of at most a third: 4 KiB payloads (×1.32), the C×D interaction (without `fsync` the p99 at 10 clients falls from 3.4 to 2.2 ms, at 160 clients it is unchanged at about 31–33 ms), the write ratio (×1.21) and `forceSync` (×0.82).

Hypotheses. **H1** holds only at low load: `forceSync` has no main effect on throughput (×1.07, interval includes no change) and lowers the write p99 by 18%, and its effect on both depends on the load (C×D). **H2** is refuted: the benefit of disabling the `fsync` does not grow with the write ratio (B×D, ×1.04, not significant). **H3** is refuted: neither the log disk nor D×F has a detectable effect. **H4** is refuted for throughput (five servers are 14% faster, not slower) and holds only in part for the p99: with 10% writes five servers raise it from 7.4 to 9.6 ms, with 50% writes they do not (A×B, ×0.85). **H5** is refuted for throughput, where the write ratio matters more than the load, and confirmed for the write p99.

6.2 One USL curve

One configuration (3 servers, write ratio 0.3), thirteen client counts from 1 to 160, three replicates each in random order, on dedicated machines, with the weighted fit and the bootstrap, Amdahl-comparison and hold-out checks described below. The effect of the number of servers is not read from separate fits but measured as factor A of Section 6.1. The model is

$$X(N) = \frac{\gamma N}{1 + \alpha(N-1) + \beta N(N-1)}, \qquad N_{\max} = \sqrt{(1-\alpha)/\beta}, \tag{3}$$

Table 4. Largest effects on throughput and on the 99th-percentile write latency: q is half the change from the low to the high level, with its 95% confidence interval and the share of the variation it explains (%). For a response analysed on $\log_{10}$, q is in $\log_{10}$ units and (×) is the factor by which the response changes from the low to the high level. †: the interval includes zero.

	throughput (ops/s), $\log_{10}$			write p99 (ms), $\log_{10}$		
Effect	q (×)	95% CI	%	q (×)	95% CI	%
C	0.149 (×1.98)	[0.133, 0.164]	25.6	0.534 (×11.68)	[0.511, 0.556]	93.7
B	-0.228 (×0.35)	[-0.244, -0.213]	60.3	0.041 (×1.21)	[0.018, 0.064]	0.6
E	-0.066 (×0.74)	[-0.081, -0.051]	5.0	0.061 (×1.32)	[0.038, 0.084]	1.2
C×D	-0.056 (×0.77)	[-0.071, -0.041]	3.6	0.059 (×1.31)	[0.036, 0.081]	1.1
A	0.029 (×1.14)	[0.014, 0.044]	1.0	0.021 (×1.10)†	[-0.001, 0.044]	0.2
D	0.015 (×1.07)†	[-0.001, 0.030]	0.3	-0.042 (×0.82)	[-0.065, -0.019]	0.6
E×F	0.021 (×1.10)	[0.006, 0.036]	0.5	-0.004 (×0.98)†	[-0.027, 0.018]	0.0
A×B	0.000 (×1.00)†	[-0.016, 0.015]	0.0	-0.034 (×0.85)	[-0.057, -0.012]	0.4
A×C	0.018 (×1.09)	[0.003, 0.033]	0.4	0.010 (×1.05)†	[-0.013, 0.033]	0.0
B×C	-0.016 (×0.93)	[-0.032, -0.001]	0.3	-0.020 (×0.91)†	[-0.043, 0.003]	0.1
B×E	0.003 (×1.01)†	[-0.012, 0.019]	0.0	-0.022 (×0.90)†	[-0.045, 0.000]	0.2

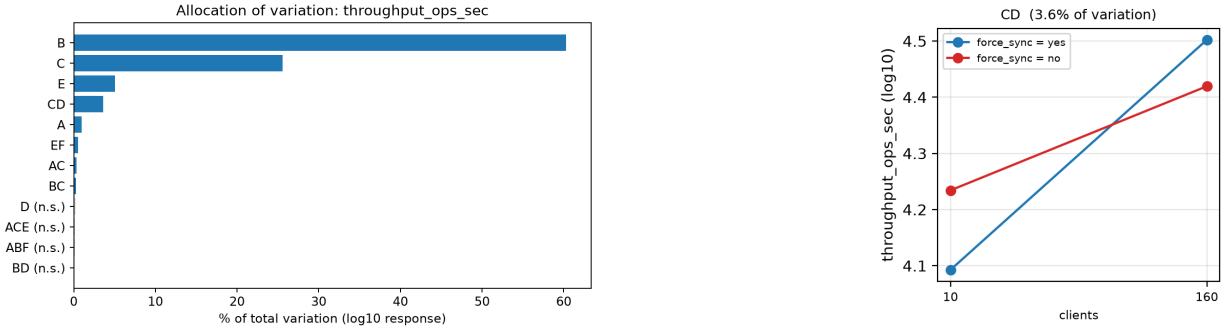


Fig. 2. Left: share of the variation of $\log_{10}$ throughput explained by each effect (n.s.: not significant). Right: the strongest interaction, clients × `forceSync`: the lines cross.

where γ is the throughput of one client, α the contention coefficient (the serialized fraction of the work) and β the coherency coefficient (the pairwise cost of keeping participants consistent, which produces retrograde scaling). The fit minimizes the relative error, so that a 3% error counts the same at 600 and at 6,000 ops/s; confidence intervals come from a bootstrap over the replicates; the full model is compared with the contention-only (Amdahl) model by AIC and an F-test, so that β is claimed only if the data justify the extra parameter; and a fit on the lower loads is used to predict the highest ones.

The replicates of a load level differed by at most 12%, and the load generator stayed below 28% CPU. Throughput rises from 2,165 ops/s with one client to 46,436 ops/s with 160 (Fig. 3). The fitted single-client throughput is γ = 2,082 ops/s (95% bootstrap interval 1,996–2,169) and the contention coefficient $\alpha = 0.0385$ (0.0345–0.0414). A coherency term is not supported: the contention-only (Amdahl) model fits as well (AIC −241.4 against −239.5 for the full USL; F-test $p = 0.72$; 37% of the bootstrap fits put β at zero), so we report α only and do not claim retrograde scaling; the peak the full USL would put at $N_{\max} \approx 469$ lies far outside the tested range and has no meaning here. With the Amdahl fit ($\alpha = 0.039$) the speed-up over one client is bounded by $1/\alpha \approx 26$ and throughput by $\gamma/\alpha \approx 53{,}500$ ops/s; at 160 clients the servers deliver 87% of that bound. The model predicts as well as it fits: fitted to $N \leq 64$ only, it predicts the throughput at 96, 128 and 160 clients within 1.9–5.1% (mean 4.0%).

Where the time of a write goes. The fit says how much of the work is serialized; ZooKeeper's per-stage counters (Section 4) say where. Table 5 gives, at four load levels of the curve, the mean time a write spent on the leader waiting for the request-preparation thread, waiting for the log-sync thread, until its batch was on disk and until a quorum had acknowledged it, and the number of requests written per `fsync` (all levels: Fig. 4 in the appendix). A stage whose waiting time grows with N while the others stay flat is the serialized stage behind α; a growing batch shows group commit amortizing the `fsync` as the load rises. From 1 to 160 clients the server-side write latency grows from 0.87 to 5.57 ms. The largest single contribution is the queue in front of the request-preparation thread, which grows from 0.02 to 1.37 ms, most of it beyond 64 clients;

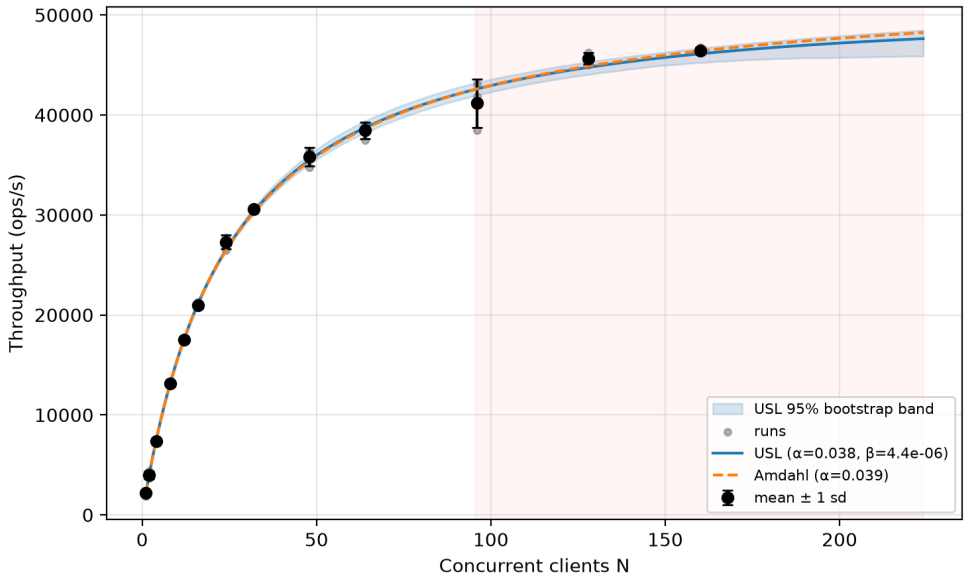


Fig. 3. Measured throughput (dots: runs; black: mean ± one standard deviation), the USL fit with its 95% bootstrap band and the contention-only (Amdahl) model. The shaded region on the right was held out for the validation.

Table 5. Time per pipeline stage on the leader (ms, mean per run, averaged over the replicates) and requests written per `fsync`, at four load levels of the USL curve; the server-side write latency is pooled over all servers. From ZooKeeper's own counters (1 ms resolution).

Leader, mean per run	$N = 1$	$N = 16$	$N = 64$	$N = 160$
prep queue (zxid) (ms)	0.02	0.05	0.35	1.37
sync queue (ms)	0.03	0.43	0.66	0.76
sync queue + flush (ms)	0.55	1.08	1.86	2.14
fsync (ms)	0.06	0.22	0.69	0.75
until quorum ACK (ms)	0.72	1.34	2.26	2.79
server-side write latency (ms)	0.87	1.61	3.26	5.57
requests per fsync	1.0	4.1	13.0	17.6

the time from the creation of a transaction until a quorum has acknowledged it grows from 0.72 to 2.79 ms, and the commit stage adds about 0.8 ms at 160 clients. The log-sync stage, in contrast, stops growing: its queue levels off at about 0.7 ms because group commit grows from 1 to 17.6 requests per `fsync`, so each write pays only a small share of an `fsync`. The serialized work behind α is therefore the leader's request pipeline (zxid assignment, proposal to the followers, commit), not the disk.

6.3 eBPF probes

Five probes observe the leader's operating system while the benchmark runs; four are our own bpftrace programs [11] and one is a bcc tool. Each is aimed at one suspected limit of Table 10.

- **fsync latency** (ours): tracepoints `syscalls:sys_enter/exit_fsync` and `fdatasync`, filtered to the ZooKeeper JVM. Number of calls and latency histogram of the log-sync stage, in microseconds. With `forceSync=no` it must see (almost) no calls, which also verifies that factor D was applied.
- **Block I/O latency** (ours, per disk): tracepoints `block_rq_issue` and `block_rq_complete`. Whether the log disk itself is slow underneath the `fsync`; compared between pd-balanced and pd-ssd. (We first used bcc's `biolatency`, but its kprobe, `blk_account_io_done`, is not traceable on the 6.1 kernel of the servers.)
- **Run-queue latency** (bcc `runqlat`, threads of the JVM only): scheduler tracepoints, from wake-up to running. Whether the server waits for a CPU, which would add to α independently of the disk.
- **TCP retransmissions** (ours): the `tcp` tracepoints `tcp_retransmit_skb` and `tcp_send_reset`. Whether packet loss adds retransmission timeouts to the round trips of the quorum.
- **CPU per pipeline thread** (ours): a 99 Hz `profile` probe filtered to the JVM and keyed by thread name. ZooKeeper runs each stage in a named thread (request preparation, log sync, commit, client

Table 6. eBPF measurements on the leader: a 45 s probe window around a run of 25 s measured (80 clients, write ratio 0.5); percentiles are upper bounds of $\log_2$ histogram buckets.

Configuration	fsync calls	fsync mean (μs)	fsync p99 (μs)	log disk p99 (μs)	run queue p99 (μs)	TCP retr.
fsync off, pd-balanced	0	–	–	16384	511	0
fsync off, pd-ssd	0	–	–	16384	511	0
fsync on, pd-balanced	21393	1064	2048	1024	511	0
fsync on, pd-ssd	23120	986	2048	512	511	0

Table 7. The same runs seen by ZooKeeper and by the thread probe: throughput with the probes and without them (mean of the runs before and after), requests written per `fsync` (per flush when it is off), ZooKeeper's own `fsync` time and time until a quorum ACK (leader means over the run), and the CPU (in cores) used by the busiest pipeline threads of the leader.

Configuration	ops/s no probes	ops/s	overhead (%)	req./fsync	ZK fsync (ms)	to quorum ACK (ms)
fsync off, pd-balanced	29276	28795	-1.7	1.9	–	1.14
fsync off, pd-ssd	28739	24992	-15.0	1.8	–	1.16
fsync on, pd-balanced	26622	28731	7.3	17.4	0.77	2.56
fsync on, pd-ssd	26674	27301	2.3	16.0	0.70	2.52

CPU cores used by	fsync off, pd-balanced	fsync off, pd-ssd	fsync on, pd-balanced	fsync on, pd-ssd
replication to followers	0.67	0.67	0.55	0.59
client connections (I/O)	0.40	0.38	0.40	0.39
PrepRequestProcessor (zxid)	0.20	0.21	0.25	0.25
CommitProcessor	0.19	0.18	0.17	0.16
SyncRequestProcessor (log, fsync)	0.10	0.09	0.10	0.11
commit workers (apply, reply)	0.06	0.05	0.06	0.06
JIT compiler	0.05	0.06	0.03	0.03
whole JVM	1.73	1.70	1.61	1.62

connections) and the JVM passes the names to Linux, so this is the CPU used by each stage; a single-threaded stage close to one full core is a serialized bottleneck.

The probes run in separate runs of a saturating configuration (3 servers, 80 clients, write ratio 0.5), at both levels of `forceSync` and of the log disk, never during the factorial runs. Every probed run is paired with an identical run without probes before it and another after it (the overhead is measured against their mean, after a discarded warm-up run), and ZooKeeper's counters are read around it (Table 7).

With `forceSync=no` the probe sees no `fsync` at all, which confirms that factor D was applied. With it, an `fsync` of the transaction log takes 1.0 ms on average on both disks (986 μs on pd-ssd, 1,064 μs on pd-balanced; most calls take 1–2 ms), although the device itself is about twice as fast on pd-ssd (I/O p99 below 0.5 against 1 ms): the device is not what makes the `fsync` slow, which is why the log disk has no effect in the factorial experiment. ZooKeeper's own counter reports 0.70–0.77 ms for the same calls because it truncates each one to whole milliseconds. Removing the `fsync` shortens the time until a quorum ACK from 2.5 to 1.15 ms and the median write from 4.8 to 3.8–3.95 ms: at this load the `fsync` is about a fifth of a write's latency. The run queue of the JVM's threads stays below 0.5 ms at the 99th percentile and there are no TCP retransmissions, so neither CPU starvation nor packet loss adds delay. The leader's JVM uses 1.6–1.7 of its two cores; the largest shares go to replication to the followers (0.55–0.67 cores) and to client connections (0.4), while the request-preparation and log-sync threads use only 0.2–0.25 and 0.1 cores. No single thread is saturated, so at this load the limit is the leader's CPU as a whole rather than one stage's thread. The probes' overhead is within the variation between runs: the two runs without probes of the same configuration differed by up to 30%. These results support H1 at moderate load (the `fsync` is on the write path and costs about 1 ms) and explain why H3 fails (the device is not the slow part of the `fsync`).

6.4 Cross-check with YCSB

As an independent check of the shape of the results we ran YCSB [9] with its ZooKeeper binding on the 3-server system of Stage I (Table 8). YCSB has a different code base and session handling, so agreement in shape tells us that a result is not a quirk of our own client; it does not validate absolute numbers, which were measured on a laptop and were not repeated.

Table 8. YCSB throughput (ops/s) by workload and thread count, Stage I laptop, 3 servers, one run per cell.

Workload	1 thread	4	16	32
A (50/50 read/update)	790	1316	1723	5964
B (95/5)	3544	6927	11436	8080
C (100% read)	4538	9074	11913	8900

Table 9. Throughput (ops/s) with and without a 20 ms egress delay, 3 servers, Stage I laptop, one run per cell.

Clients	1	5	10	20	40	80
No delay	2386	7016	11680	14832	16238	16255
20 ms delay	20.6	108.9	216.7	434.6	865.5	1709
Ratio	116×	64×	54×	34×	19×	9.5×

At 16 threads the read-heavy workloads B and C reach 6.6× and 6.9× the throughput of the 50/50 workload A, and the average update latency is 3–4.5× the read latency (3713 against 1078 μs in A), consistent with the extra steps of the write pipeline. B and C peak at 16 threads and decline at 32. Workload A jumps 3.5× between 16 and 32 threads (1723 to 5964), which we cannot explain from the data; with a single run per cell, warm-up and garbage collection are possible causes.

6.5 Leader failover

With a continuous benchmark running against a 3-server system we identified the leader with the `stat` command and killed its container. The two survivors kept a majority and continued serving, and ZooKeeper logged the re-election: `FOLLOWING - LEADER ELECTION TOOK - 211 MS` in the first run, and 216 ms (follower) and 235 ms (new leader) in a repetition. This is ZooKeeper's own measurement of the election phase, not the outage seen by a client, which also waits for its retry and reconnect logic and which we did not instrument.

6.6 Network delay

To see how the model behaves under a large increase of the service time per request, we injected a 20 ms±2 ms delay with Pumba [10], which drives `tc netem`, on the egress of every server container of the Stage I servers (the delay is applied inside each container's network namespace, because a delay on the Docker bridge does not affect traffic between containers) and re-ran the sweep with the corrected client (Table 9).

With one client throughput falls 116×, but at 80 clients it falls only 9.5×. Throughput per client is constant at about 21.5 ops/s, one operation every 46.5 ms: a closed-loop client completes one operation per round trip, and N clients in parallel complete N operations as long as the server is not busy. The fitted α and β are therefore near zero: the delay hides the server, and the saturation point has moved beyond the loads we tested. α and β describe the regime that was measured and are not constants of the system. The latencies give a direct view of the write pipeline: the median read takes 22.7 ms, one delayed hop (the reply), and the median write takes 106 ms, about five delayed hops (forward to the leader, proposal, acknowledgment, commit, reply), which is what Figure 1 predicts for a client attached to a follower.

7 Discussion: Which Parts of the Pipeline Limit Scalability

Each term of the USL corresponds to a part of the write pipeline (Table 10). Reads are different: a read is answered by the server the client is attached to without any coordination, so reads are limited by that server's CPU and not by the leader. In the curve of Section 6.2, N is the number of clients and the number of servers is three, so the cost of the quorum per write (one proposal to each follower, a majority of acknowledgments, a commit) does not change with N: it is part of γ, and of α where it is serialized, but it is not what β measures. The effect of the number of servers is measured separately, as factor A of the factorial experiment, and appears in the last row of the table.

The measurements point to one dominant limit, and it is not the disk: the log disk has no effect, an `fsync` costs about 1 ms whatever the device, and group commit spreads it over up to 17 requests, so the log-sync stage's queue stops growing beyond about 64 clients. What keeps growing is the queueing in front of the leader's request pipeline, and the leader's CPU is mostly busy replicating to the followers and serving

Table 10. USL terms (and the number of servers), the pipeline stage each one points to, the evidence of this work and a mitigation.

Term	Pipeline stage	Evidence	Mitigation
α	Serialized stages of the leader: request preparation (one thread assigns every zxid), the log-sync stage (one thread appends and `fsync`s, a batch at a time) and the commit stage	$\alpha = 0.039$; the prep-queue time grows from 0.02 to 1.37 ms (Table 5); the leader's JVM uses 1.6–1.7 of its 2 cores, mostly replication and client I/O (Table 7); D and F have no main effect on throughput	More CPU for the leader, or less work on it (`leaderServes=no`); fewer, larger writes by the clients (multi-operations); not a faster log disk
β	Interference between concurrent requests that grows with pairs of clients: in the commit stage a committed write waits for the reads in progress; per-session work on the servers; garbage collection	Not supported (AIC; F-test $p = 0.72$); the commit-stage time grows only linearly, to about 0.8 ms at 160 clients	Fewer, longer-lived sessions; separate read and write clients; *observers* to take reads off the voting servers
γ	Cost of one request in isolation: network round trips (forward, proposal, acknowledgment, commit), serialization, payload bytes, the `fsync` of a single write	$\gamma = 2{,}082$ ops/s, i.e. 0.48 ms per request for one client; 4 KiB payloads cost 26% of throughput (E); 0.72 ms until quorum ACK at $N = 1$	Asynchronous clients that pipeline several requests per session; smaller payloads
Servers (factor A)	Quorum size: each proposal goes to every follower and the leader waits for a majority	Five servers are 14% faster ($\times 1.14$) at both write ratios; with 10% writes their write p99 is higher (A$\times$B); no TCP retransmissions	Keep the number of servers small; observers to scale reads without enlarging the quorum; hierarchical quorums

client connections; the contention coefficient $\alpha = 0.039$ is the share of each request that passes through this serialized path. The mitigations the data support are therefore those that relieve the leader rather than the disk: more CPU for the leader, or less work on it, since the leader can be configured not to accept clients and to focus on coordination (`leaderServes=no`) [4]; observers to add read capacity without enlarging the quorum (more servers already raised throughput by 14%); and fewer, larger writes from the clients (multi-operations), which amortize the per-request replication work the way group commit amortizes the `fsync`. A faster log disk would not help, and disabling the `fsync` lowers throughput at high load.

8 Conclusion

Insights. (1) The workload, more than the configuration, decides ZooKeeper's throughput: the share of writes explains 60% of its variation and the load 26%, while the log disk and `fsync` explain almost nothing on average; for the write tail latency the load explains 94%. (2) The scalability curve of one configuration is described by contention alone ($\alpha = 0.039$, $\gamma = 2{,}082$ ops/s; a coherency term is not supported up to 160 clients): throughput levels off towards about 53,500 ops/s instead of falling, and a fit on up to 64 clients predicts the higher loads within 5%. (3) ZooKeeper's per-stage counters and the probes locate the contention in the leader's request pipeline and its CPU, not in the disk: an `fsync` costs about 1 ms on either disk and group commit amortizes it over up to 17 requests, which is also why disabling it helps only at low load and costs 17% of throughput at high load. Two methodological lessons stand out as well. First, a load generator must be validated before its numbers are interpreted: in Stage I a client that logged every operation produced a contention coefficient that looked like a property of ZooKeeper and was not, and the check that exposed it was counting the lines of output of the client. Second, on shared hardware a single run per point cannot separate an effect from drift (identical runs differed by up to a factor of two), so every conclusion in this report rests on replicated, randomized runs on dedicated machines. *Limitations.* All servers are in one zone, so the round trips of the quorum are those of one data centre and not of a geo-replicated deployment; the virtual machines share their hosts with other tenants, which replicates and randomization average out but do not remove; the hard-to-change factors (A, D, F) are tested against the pooled error rather than a whole-plot error; 20 GB persistent disks on two-vCPU machines tie the disk results to this testbed; the USL curve covers one configuration (three servers, 30% writes, 128-byte payloads); and the workload has no creates, deletes, watches or multi-operations. Natural next steps are observers, asynchronous clients and a multi-zone placement.

A Sign table

Table 11. Sign table of the 2^{6-1} design (run × factor, ±1) with the mean throughput of the two replicates.

Run	A	B	C	D	E	F	throughput_ops_sec
1	−	−	−	−	−	−	25667
2	+	−	−	−	−	+	25289
3	−	+	−	−	−	+	8804
4	+	+	−	−	−	−	9702
5	−	−	+	−	−	+	53235
6	+	−	+	−	−	−	79861
7	−	+	+	−	−	−	19988
8	+	+	+	−	−	+	20315
9	−	−	−	+	−	+	29633
10	+	−	−	+	−	−	32421
11	−	+	−	+	−	−	12408
12	+	+	−	+	−	+	11947
13	−	−	+	+	−	−	49811
14	+	−	+	+	−	+	60824
15	−	+	+	+	−	+	12908
16	+	+	+	+	−	−	23112
17	−	−	−	−	+	+	16841
18	+	−	−	−	+	−	16902
19	−	+	−	−	+	−	5664
20	+	+	−	−	+	+	6245
21	−	−	+	−	+	−	45392
22	+	−	+	−	+	+	54084
23	−	+	+	−	+	+	15242
24	+	+	+	−	+	−	16408
25	−	−	−	+	+	−	22213
26	+	−	−	+	+	+	26116
27	−	+	−	+	+	+	9549
28	+	+	−	+	+	−	9646
29	−	−	+	+	+	+	38703
30	+	−	+	+	+	−	39773
31	−	+	+	+	+	−	12076
32	+	+	+	+	+	+	15022

B Per-stage times at every load level

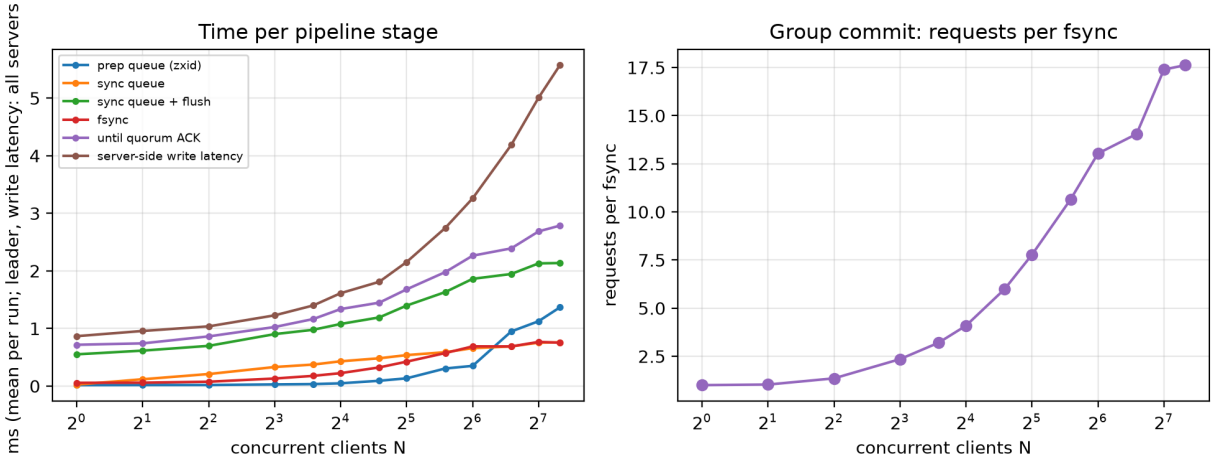


Fig. 4. Time per pipeline stage (left) and requests written per `fsync` (right) against the number of clients, for every load level of the USL curve: leader means per run, averaged over the replicates (the server-side write latency is pooled over all servers).

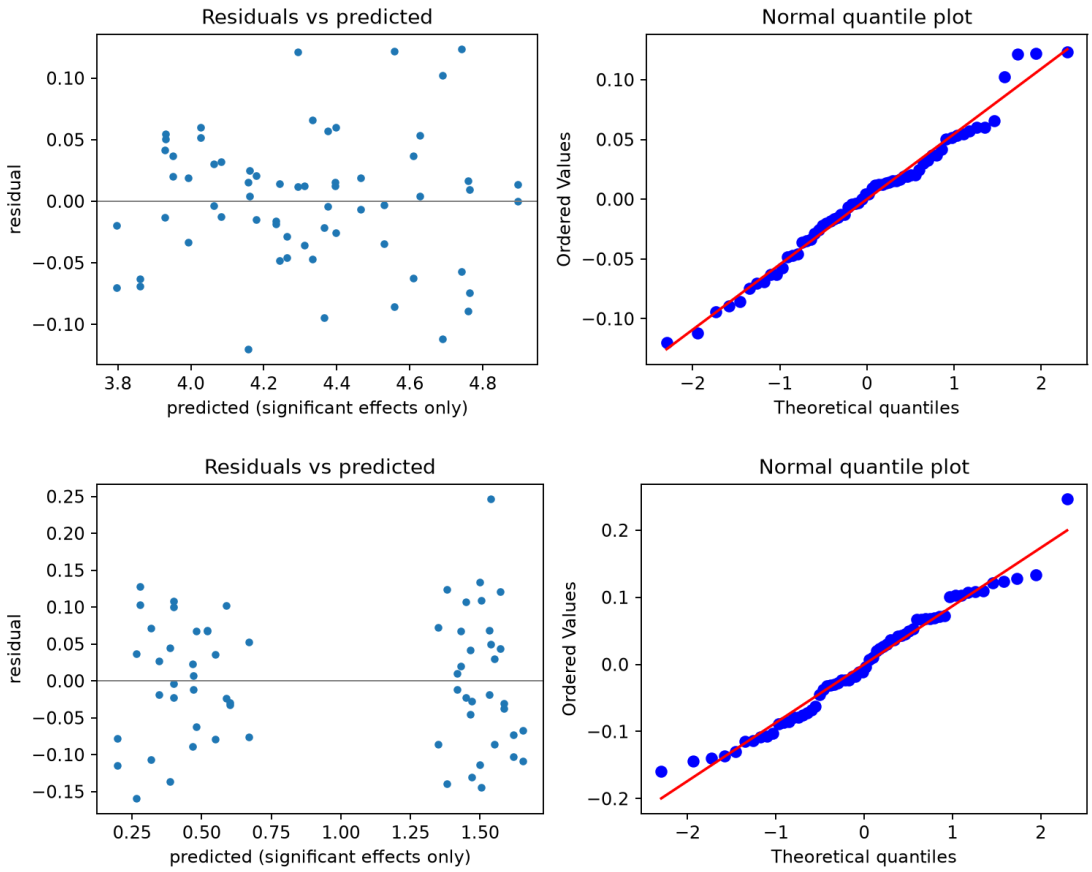


Fig. 5. Residuals of the effects model of the factorial experiment against the predicted value (left) and on a normal quantile plot (right): throughput (top) and 99th-percentile write latency (bottom), in the model used in Table 4.

C Model checks

References

1. P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for Internet-scale systems. In *USENIX Annual Technical Conference*, 2010.
2. F. P. Junqueira, B. C. Reed, and M. Serafini. Zab: High-performance broadcast for primary-backup systems. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2011.
3. Apache Software Foundation. ZooKeeper: A Distributed Coordination Service for Distributed Applications. ZooKeeper 3.9.6 documentation, 2024. `https://zookeeper.apache.org/doc/r3.9.6/zookeeperOver.html`.
4. Apache Software Foundation. ZooKeeper Administrator's Guide. ZooKeeper 3.9.3 documentation. `https://zookeeper.apache.org/doc/r3.9.3/zookeeperAdmin.html`.
5. MIT 6.824: Distributed Systems. ZooKeeper. Lecture 8, Spring 2020. `https://www.youtube.com/watch?v=pbmyrNjzdDk`.
6. Distributed Systems Conference. Applying The Universal Scalability Law to Distributed Systems by Dr. Neil J. Gunther. DSConf 2019, March 16, 2019. `https://www.youtube.com/watch?v=cuieNj3NrZ8`.
7. N. J. Gunther. *Guerrilla Capacity Planning*. Springer, 2007.
8. R. Jain. *The Art of Computer Systems Performance Analysis*. Wiley, 1991.
9. B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking cloud serving systems with YCSB. In *ACM Symposium on Cloud Computing (SoCC)*, 2010.
10. A. Ledenev. Pumba: chaos testing and network emulation for Docker containers. `https://github.com/alexei-led/pumba`.
11. B. Gregg. *BPF Performance Tools*. Addison-Wesley, 2019.